

AGGIESHADE

Development Plan & Technical Proposal

A campus routing system that helps pedestrians trade a small amount of travel time for substantially less direct sun exposure.

Core promise

Enter two campus locations.
Compare the fastest route with a time-aware, more shaded alternative.

Technical thesis

Combine pedestrian graph routing, public GIS, solar geometry, and computational geometry - no ML required for the MVP.

Delivery target

A demonstrable v1.0 in 10-12 weeks, with a usable conventional router by the Week 5 checkpoint.

Prepared as a student-project technical proposal | August 2026

1. Project overview & MVP

Problem. College Station heat makes short campus walks uncomfortable, yet conventional navigation optimizes mainly for distance or time. AggieShade makes sun exposure a first-class route cost.

MVP. A user selects an origin and destination, optionally uses current location, chooses a preference from Fastest to Most Shade, and receives at least two comparable routes with ETA, distance, and estimated percent shaded.

Fastest	Balanced	Shadiest
8 min 0.42 mi 31% shaded	9 min 0.45 mi 56% shaded	10 min 0.48 mi 72% shaded +2 min

Success means the app can explain the tradeoff clearly: “two extra minutes buys 41 percentage points more shade.” Accounts, reviews, social features, AI, live weather, and crowdsourcing remain outside the initial scope.

Key product metrics

- Route correctness: valid, connected, walkable paths between major campus destinations.
- Shade usefulness: predicted shaded portions broadly match field observations at selected test times.
- Decision clarity: users can compare time, distance, and shade without interpreting technical data.
- Performance: route response feels interactive; expensive shadow work is cached or precomputed.

2. Recommended technology stack

Layer	Recommendation	Role
Mobile	React Native + Expo	One TypeScript codebase; GPS, search, and route map
Map	MapLibre (or Mapbox)	Campus basemap, polylines, shade overlays
API	Python + FastAPI	Routing requests, validation, service orchestration
Geospatial	GeoPandas, Shapely, PyProj	Projection, buffering, intersection, shadow polygons
Graph	NetworkX initially	A*/Dijkstra, graph attributes, rapid iteration
Storage	PostgreSQL + PostGIS	Spatial layers, graph metadata, cached edge scores
Solar	pvlb or Astral	Solar altitude and azimuth by place and time
Data jobs	Python scripts / scheduled workers	Download, normalize, build graph, precompute shade

Why this stack

The mobile layer stays productive and cross-platform while Python provides a mature geospatial toolchain. PostGIS becomes the durable source of spatial truth. NetworkX is appropriate for a campus-scale prototype; a dedicated routing service can replace it only if performance evidence demands that change.

3. Data foundation & pedestrian graph

TAMU public ArcGIS services are the project's main technical advantage. Relevant layers described in the source plan include university buildings, trees, sidewalks, steps, ramps/curb cuts, campus driveways, and roads. Tree records include location, species, height, canopy spread, health, and status; buildings provide footprints and floor counts.

Data-source strategy

- Use OpenStreetMap pedestrian ways as the initial routable centerline graph.
- Overlay official TAMU tree and building geometry for shade; use sidewalk and accessibility layers for validation and later graph refinement.
- Record source date, layer version, units, coordinate reference system, and quality flags during ingestion.
- Project all geometry into a local metric CRS before measuring distance, buffering, or casting shadows.

Graph design

Intersections, entrances, and path endpoints become nodes. Walkable segments become directed edges. Each edge stores geometry and routing attributes such as length, travel time, slope/accessibility when available, surface, data quality, and a time-indexed shade score.

```
node: id, lat, lon, type
edge: from, to, geometry, length_m, walk_time_s,
      shade_ratio[time_bucket], exposure_m, accessibility
```

Snap user-selected buildings to verified entrances or nearby walkable nodes - not merely to the building centroid. Validate topology for disconnected components, accidental road crossings, duplicate edges, stairs, and restricted areas.

4. Solar and shade modeling

At request time, the shade engine computes solar altitude and azimuth for College Station at the selected date and time. For an object height h and solar altitude α , an initial shadow-length approximation is:

$$L = h / \tan(\alpha)$$

The shadow extends opposite the solar azimuth. When the sun is below the horizon, daylight routing should treat the route as non-sun-exposed rather than generating unbounded shadows.

Trees Approximate each canopy as a circle or ellipse. Radius starts at canopy spread / 2. Translate/project the canopy opposite the sun using tree height, then optionally apply a transmissivity factor for imperfect foliage.	Buildings Extrude/project the footprint opposite the sun. When exact elevation is missing, estimate height as floors x 3.5 m and mark the estimate with lower confidence.	Combined shade Union tree and building shadow polygons for the selected time bucket. Keep provenance so field tests can distinguish building error from canopy error.
--	---	---

Practical accuracy controls

- Use 10- or 15-minute time buckets and cache results; interpolate only if testing shows value.
- Clip implausibly long shadows near sunrise/sunset and surface low-confidence predictions.
- Account for tree health/status and future seasonal canopy factors; avoid claiming physical ray-tracing accuracy.
- Create a small ground-truth set: photographed or observed shaded/unshaded segments at multiple times.

5. Edge shade scoring & routing

Sample points along each edge every 2-3 meters, or compute the exact geometric intersection length. The baseline shade ratio is the shaded length divided by total edge length:

$$\begin{aligned}\text{shade}(e,t) &= \text{shaded_length}(e,t) / \text{length}(e) \\ \text{exposure}(e,t) &= \text{length}(e) \times [1 - \text{shade}(e,t)]\end{aligned}$$

A simple user-controlled objective is:

$$w(e,t) = \text{distance}(e) + \text{lambda} \times \text{exposure}(e,t)$$

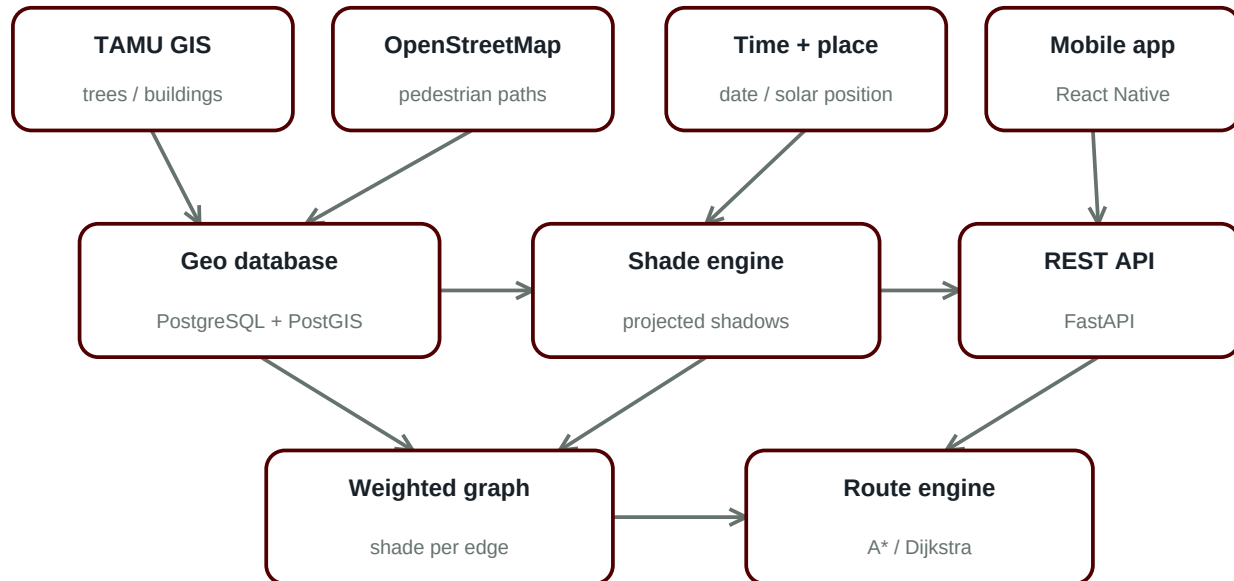
Set lambda = 0 for fastest, about 1 for balanced, and a higher calibrated value (for example 3-5) for shadiest. Dijkstra works directly. For A*, use an admissible lower-bound heuristic such as straight-line distance, since the exposure penalty is nonnegative.

Route-level reporting

- Distance = sum of edge lengths; ETA = sum of walking times.
- Shade percent = total shaded length / total route length - length-weighted, not an average of edge percentages.
- Return distinct alternatives. If the “shadiest” path is effectively identical, do not present a fake choice.
- Constrain detours (for example, no more than 25-35% beyond fastest) so maximum shade stays useful.

Later refinements can include UV-weighted exposure, temperature, cloud cover, accessibility, construction closures, and multi-objective Pareto routing.

6. System architecture



The offline pipeline ingests and normalizes spatial sources, builds the walk graph, and prepares time-bucketed shadow/edge data. The online API snaps endpoints, selects the relevant time bucket, applies the user's lambda, runs A*, and returns route geometry plus comparison metrics to the mobile client.

API sketch

```

POST /routes
{ origin, destination, departure_time, shade_preference, accessibility? }

-> fastest_route, recommended_route, alternatives[]
-> each: geometry, distance_m, duration_s, shade_ratio, confidence
  
```

7. UI concepts

Search & preference Two location fields, current-location shortcut, departure time (“leave now” by default), and one plain-language preference control.	Map & comparison Emphasize one recommended route while keeping fastest visible. Use distinct line styles/colors and a clear legend; do not rely on color alone.	During navigation Show next instruction, remaining time, off-route recovery, and a compact shade estimate. Avoid distracting live geometry details.
---	---	---

AGGIESHADE From Current location To Zachry Engineering Fastest ----o----- Most shade [Find route]	EVANS -> ZACHRY route map + shade overlay <table><tr><td>Recommended</td><td>10 min</td><td> </td><td>72% shade</td></tr><tr><td>Fastest</td><td>8 min</td><td> </td><td>31% shade</td></tr></table> +2 min for +41 pts shade	Recommended	10 min		72% shade	Fastest	8 min		31% shade
Recommended	10 min		72% shade						
Fastest	8 min		31% shade						

Accessibility and trust

Use readable contrast, scalable text, explicit units, and accessible route filters when data permits. Label estimates as estimates, include the departure time used, and provide a small “Why this route?” explanation tied to time and shade.

8. Ten-to-twelve-week roadmap

Week	Outcome	Exit criterion
1	Requirements, user flows, repository	MVP scope and acceptance tests agreed
2	Mobile map, permissions, GPS	Campus map opens; current location works
3	Building/place search	Major destinations resolve to map points
4	Pedestrian graph pipeline	Connected campus graph persisted
5	Conventional A*/Dijkstra routing	Zachry-to-Evans route renders end-to-end
6	TAMU GIS ingestion	Trees/buildings normalized and queryable
7	Solar-position service	Altitude/azimuth verified for test times
8	Tree shadow model	Canopy shadows render for a time bucket
9	Building shadow model	Footprint shadows render; heights flagged
10	Edge scores + shade routing	Fastest/balanced/shadiest compare correctly
11	UX polish + navigation	Metrics, errors, rerouting, accessibility QA
12	Field validation + deployment	Campus test report and demo release

Week 5 is the critical checkpoint: the product must already route between two campus buildings. Shade features improve a working navigator; they do not postpone it.

9. Version milestones & codebase

v0.1 - Navigator Campus map, GPS, destination search, ordinary walking route, baseline deployment.	v0.5 - Shade prototype Imported GIS, static or representative-time shade, edge scoring, shade-aware route.	v1.0 - Dynamic experience Date/time solar position, dynamic tree/building shadows, route alternatives, shade percentage, field-tested calibration.
--	--	--

Recommended repository structure

```
AggieShade/  
  mobile/      components/ screens/ maps/ api/  
  backend/  
    routing/   graph.py  astar.py  weights.py  
    shade/     solar.py  trees.py  buildings.py  shadows.py  
    gis/       tamu.py   osm.py  
    models/    request and persistence models  
  scripts/     download_tamu_gis.py  build_graph.py  calculate_shade.py  
  tests/       routing/ geometry/ api/ field_cases/  
  docs/        data dictionary, ADRs, validation notes
```

Keep ingestion, geometry, routing, and client concerns separate. Store configuration - CRS, floor-height assumption, sampling distance, time-bucket size, and lambda presets - outside algorithm code and document changes in lightweight architecture decision records.

10. Key decisions, risks & next action

Decision	Recommendation / rationale
Routing base	Start with OSM pedestrian centerlines; overlay and validate against TAMU layers. Avoid deriving perfect centerlines from polygons in the first release.
Shade technique	Use computational geometry, not ML. It is explainable, testable, and supported by existing height/canopy/footprint data.
Time model	Use cached 10-15 minute buckets before attempting continuous real-time recomputation.
Building height	Estimate from floor count only when necessary; carry a confidence flag.
Optimization	Use distance plus lambda-weighted sun exposure, with a detour cap and distinct route alternatives.
Validation	Field-test selected corridors across morning, midday, and afternoon; measure both route validity and shade agreement.

Primary risks

- Data freshness or licensing/terms: capture metadata and confirm permitted production use.
- Topology errors: automated connectivity tests plus field verification of high-traffic corridors.
- Shade uncertainty: confidence indicators, calibrated assumptions, and modest product claims.
- Battery/latency: precompute, cache, simplify geometries, and avoid unnecessary client-side spatial work.

First coding milestone
Launch a React Native campus map, select two TAMU buildings, request an ordinary walking route from FastAPI, and draw the returned path. Complete this vertical slice before building the shade engine.

Source note: This proposal is based on the complete referenced AggieShade planning conversation. Before implementation, verify current TAMU ArcGIS layer URLs, schemas, update dates, and usage terms; verify current OpenStreetMap coverage and map-provider licensing.